

# Scan provider configuration

---

The backend uses the Gemini Developer API via the Google Generative AI SDK, not Vertex AI OAuth authentication.

## Gemini setup

1. Create a Gemini Developer API key in Google AI Studio for the intended project. Ensure the project has API access, quota, and billing if required by the chosen model.
2. Set `GEMINI_API_KEY` in the backend environment (locally, `backend/.env`). Use the API key itself, not an OAuth access token, a service-account JSON document, or a `Bearer ...` value. Do not put it in a frontend `EXPO_PUBLIC_` variable.
3. Optionally set `GEMINI_MODEL` to a model available to that project. The default is `gemini-2.5-flash`. If Google retires this model, select a currently available compatible model.
4. Environment loading now resolves [the backend environment file](#) relative to the backend module, independent of the launch directory. Restart the backend after editing configuration. Existing process/deployment environment variables take precedence over dotenv values; check the hosting secret if a local edit appears ineffective.
5. Check key API restrictions, project selection, and model access if authentication still fails. `ACCESS_TOKEN_TYPE_UNSUPPORTED` is treated as a provider configuration failure; it is not an expired LeafCheck user session.

Credentials are read when creating the scan model rather than at route import time. The SDK supplies the key in its API-key header. No manual bearer header is added. Never paste real keys into logs, tickets, screenshots, or source control.

## Other providers and expected outcomes

- `PLANTNET_API_KEY`: required for identification. A provider 404 or empty species list becomes a controlled no-plant response (422), before plant creation or Gemini generation.
- `PERENIAL_API_KEY`: optional enrichment. Scientific-name and common-name searches are bounded by a shared timeout. Results must confirm the scientific identity before caching; no trustworthy match means diagnosis proceeds without reference specs. The Gemini prompt explicitly discloses this limitation.
- Gemini missing/invalid credentials or upstream authentication rejection return `SCAN_AI_CONFIGURATION` with 503. Rate limits return `SCAN_AI_BUSY`; other generation failures return `SCAN_AI_UNAVAILABLE`. Raw provider responses and credential-bearing URLs are not returned or logged by the scan error handler.

Automated provider tests mock network responses. They do not verify the validity of production credentials, available model quota, or live provider availability.

## Startup validation and diagnosis

Both mandatory provider keys are validated before the server listens. Gemini must use a standard Google API key: **Alza** followed by 35 URL-safe characters. Missing or malformed keys cause an actionable, secret-free startup message and exit status 1. Format validation does not establish authorization, API restrictions, quota

or model availability. Runtime validation remains active in both AI routes to catch configuration changes after startup.

See [the diagnosis and data-flow report](#) for the reproduced upstream failure, implementation steps, and deployment remediation.